

UT.2 PROGRAMACIÓN FUNCIONAL

1. INTRODUCCIÓN	2
1.1 CLASES ANÓNIMAS	4
2. Java lambdas Y SIMPLIFICACIONES	5
3. Java FUNCTIONAL INTERFACE Y EXPRESIONES LAMBDA	6
4. METHOD REFERENCES	11
4.1 REFERENCIA A UN MÉTODO ESTÁTICO (Clase::metodoEstatico)	11
4.2 REFERENCIA A UN MÉTODO DE INSTANCIA DE UN OBJETO PARTICULAR (objeto::metodoInstancia)	12
4.3 REFERENCIA A UN MÉTODO DE INSTANCIA DE UN OBJETO ARBITRARIO DE UN TIPO PARTICULAR (Clase::metodoInstancia)	13
4.4 REFERENCIA A UN CONSTRUCTOR (Clase::new)	13
5. PRINCIPALES INTERFACES FUNCIONALES DE LA API Java	14
5.1 Predicate<T>	15
5.2 Function<T, R>	15
5.3 Consumer<T>	15
6. LA API Stream	17
6.1 CREACIÓN DEL OBJETO Stream	17
6.2 OPERACIONES INTERMEDIAS	18
6.3 OPERACIÓN TERMINAL	18
6.4 EJEMPLOS	18

1.INTRODUCCIÓN

¿Qué es una Java lambda? . Esta es una de las preguntas más habituales que se hacen todos los programadores cuando comienzan a trabajar en Java . La programación funcional no es un concepto sencillo de entender, por lo que vamos a explicarlo a través de varios ejemplos apoyándonos en el concepto de ordenación de colecciones . Para ello usaremos de entrada un ejemplo de Java Comparator que use el concepto de clase Persona que contiene como propiedades el nombre, los apellidos y la edad.

```
package ejemplos.ejemplo01;
public class Persona {
    private String nombre;
    private String apellidos;
    private int edad;

    public Persona() {};

    public Persona(String nombre, String apellidos, int edad) {
        super();
        this.nombre = nombre;
        this.apellidos = apellidos;
        this.edad = edad;
    }
    public String getNombre() {
        return nombre;
    }
    public void setNombre(String nombre) {
        this.nombre = nombre;
    }
    public String getApellidos() {
        return apellidos;
    }
    public void setApellidos(String apellidos) {
        this.apellidos = apellidos;
    }
    public int getEdad() {
        return edad;
    }
    public void setEdad(int edad) {
        this.edad = edad;
    }
    @Override
    public String toString() {
```

```

        return "Persona [nombre=" + nombre + ", apellidos="
+ apellidos + ", edad=" + edad + "];"
    }
}

```

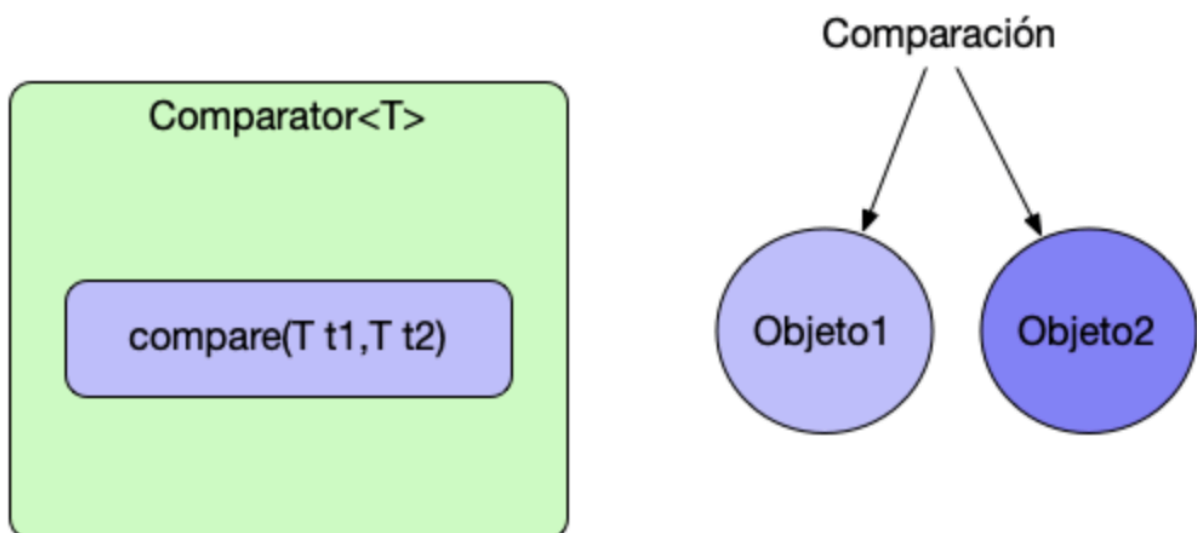
Una vez que hemos construido la clase Persona, vamos a construir un PersonaEdadComparator que se encarga de comparar las personas por edad, permitiéndonos su posterior ordenación dentro de una lista. Para ello deberemos obligar a la clase a implementar o el interfaz Comparable o el interfaz Comparator, que es de tipo genérico. En este caso optamos por la segunda opción.

```

import java.util.Comparator;
public class PersonaEdadComparator implements
Comparator<Persona> {
    @Override
    public int compare(Persona p1, Persona p2) {
        return Integer.compare(p1.getEdad(), p2.getEdad());
    }
}

```

Recordemos que el interface Comparator solo contiene un método compare que recibe dos parámetros y nos devuelve un valor numérico dependiendo si un objeto es mayor que otro, igual o menor a nivel de la propiedad o conjunto de propiedades que comparamos con nuestra clase. En este caso estamos comparando por edades usando el método estático compare de la clase Integer.



Una vez tenemos el comparador construido, el siguiente paso es construir un programa Main que construya una lista de Personas y se encargue de ordenarlas por edad:

```

import java.util.ArrayList;
import java.util.List;

```

```

public class Principal {
    public static void main(String[] args) {
        Persona personaA = new Persona("pedro", "perez",
20);

        Persona personaB = new Persona("ana", "blanco", 15);
        Persona personaC = new Persona("miguel", "alvarez",
50);

        List<Persona> lista = new ArrayList<>();
        lista.add(personaA);
        lista.add(personaB);
        lista.add(personaC);
        lista.sort(new PersonaEdadComparator());
        for (Persona p : lista) {
            System.out.println(p.getNombre() + " , " +
p.getEdad());
        }
    }
}

```

Como puedes ver, en el programa se han generado varios objetos Persona y se han añadido a la lista. Posteriormente se invoca el método sort del interfaz List, que recibe un objeto de tipo Comparator y se encarga de comparar cada uno de los elementos de la lista. En nuestro caso hemos usado PersonaEdadComparator y nos ordenará por edad:

```

<terminated> Principal [Java Applica
ana ,15
pedro ,20
miguel ,50

```

1.1 CLASES ANÓNIMAS

Una pregunta que suele plantearse es si para ordenar una lista de elementos hace falta crear una nueva clase por cada tipo de ordenación, ya que no es algo que resulte cómodo y genera una explosión de clases a nivel de código. La respuesta es que no es obligatorio, y que es posible usar una clase anónima para este fin:

```

import java.util.ArrayList;
import java.util.Comparator;
import java.util.List;

```

```

import ejemplo01.Persona;
public class Principal {
    public static void main(String[] args) {
        Persona personaA = new Persona("pedro", "perez",
20);

        Persona personaB = new Persona("ana", "blanco", 15);
        Persona personaC = new Persona("miguel", "alvarez",
50);

        List<Persona> lista = new ArrayList<>();
        lista.add(personaA);
        lista.add(personaB);
        lista.add(personaC);
        lista.sort(new Comparator<Persona>() {
            @Override
            public int compare(Persona p1, Persona p2) {
                return Integer.compare(p1.getEdad(),
p2.getEdad());
            }
        });
        for (Persona p : lista) {
            System.out.println(p.getNombre() + " ," +
p.getEdad());
        }
    }
}

```

Esta clase se define dentro de nuestro propio código y no puede ser reutilizada en ningún otro lugar ya que **solo se aplica a una situación muy muy concreta**.

Realmente, si nos ponemos a revisar el código, nos daremos cuenta de que solo la condición de la comparación de edades es relevante. El resto del código es superfluo y no aporta gran cosa.

Ahora bien, al estar Java tan basado en el manejo de clases, este código era totalmente obligatorio hasta la versión de Java 7.

2. Java lambdas Y SIMPLIFICACIONES

La realidad es que tenemos mucho código que parece que aporta poco , sin embargo recordemos que si estamos todavía bajo el paraguas de Java 7 , **la sintaxis es obligatoria**. ¿Cómo podemos simplificarla con Java 8? . **Podemos hacer uso de expresiones lambda y eliminar gran parte del código:**

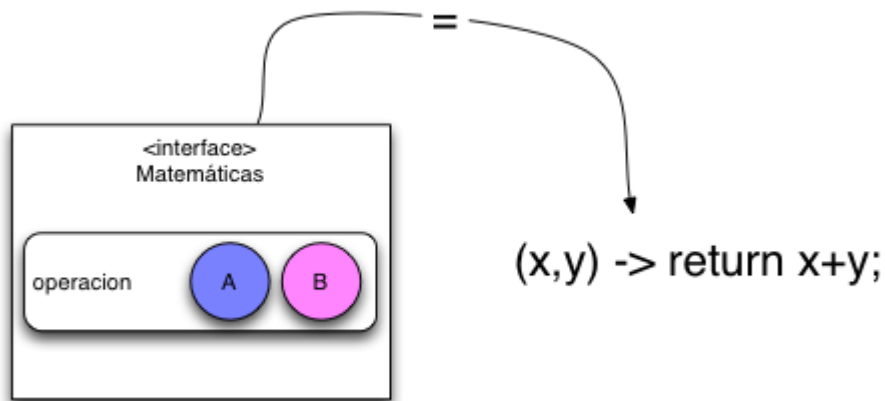
En este caso lo que haremos es eliminar la clase anónima y quedarnos únicamente con lo que se denomina un bloque lambda, el cual implementa la funcionalidad de una manera

funcional . Es decir define una función dentro de nuestro código con sus parámetros de entrada y retorno. Si mostramos el código las cosas quedan mucho más sencillas.

3. Java FUNCTIONAL INTERFACE Y EXPRESIONES LAMBDA

Java8 incluye muchas novedades, y entre ellas destacan las expresiones lambda. Podremos utilizar una expresión lambda para definir el comportamiento de una función o método.

Es aquí donde aparece el concepto de Java Functional Interface. Un interfaz funcional es aquella interfaz que solo dispone de un método abstracto. Recordemos que a día de hoy los interfaces han evolucionado mucho y pueden disponer de métodos por defecto. Así pues vamos a igualar nuestra expresión lambda a un interfaz funcional.



Como hemos visto, hasta ahora para implementar la interfaz teníamos que crear el código de la clase completo. Con la expresión lambda no desharemos de todo el código superfluo que no aporta nada, y nos quedaremos con la implementación del único método abstracto de la expresión reducido a su mínima expresión. Veámoslo a través de un ejemplo.

Comencemos creando una interfaz funcional (se utiliza la anotación `@FunctionalInterface`, que obliga al desarrollador a solo incluir un método abstracto. Respetar esto es fundamental: Si más adelante, por error, se intenta añadir un segundo método a esa interfaz, el compilador lanzará un error inmediatamente, evitando que se rompan de forma silenciosa todas las lambdas del programa que dependían de ella.):

```
@FunctionalInterface
public interface Matematica {
    int operacion(int a, int b);
}
```

La interfaz tiene un único método que recibe dos parámetros enteros y devuelve otro entero. Ahora hay que implementar el cuerpo del método en una clase:

```
public class Sumar implements Matematica {
    public int operacion(int a, int b) {
        return a + b;
    }
}
```

Por último utilizamos la clase en un programa:

```
public class Programa {
    public static void main(String[] args) {
        Matematica sumar = new Sumar();
        System.out.println(sumar.operacion(3, 4));
    }
}
```

Ahora nos planteamos las siguientes preguntas:

- Si la interfaz funcional solamente tiene un método abstracto, solo ese método podrá ser implementado. ¿Para qué necesitamos su nombre?
- Si la interfaz fija el tipo de los parámetros que recibe, ¿Para qué indicarlos al implementar el método? El compilador deduce automáticamente los tipos de los parámetros observando el contexto, ya sea mirando el lado izquierdo de la asignación (el tipo de la variable) o la firma del método que está recibiendo nuestra lambda (este proceso es conocido como inferencia de tipos).
- Yendo más allá, si vamos a devolver directamente una operación sobre los atributos (solo tenemos la línea con el return), ¿Para qué necesitamos la palabra return? Si hay una sola línea podemos suponer que es lo que queremos devolver (el tipo devuelto tendrá que coincidir, eso sí, con el tipo declarado en la interfaz)

Veamos cómo queda entonces nuestro código. Pasamos de crear ninguna clase e implementamos directamente el método abstracto a través de una lambda:

```
public class Programa {
    public static void main(String[] args) {
        Matematica sumar = (a, b) -> a + b;
        System.out.println(sumar.operacion(3, 4));
    }
}
```

La referencia sumar es de tipo Matematica (la interfaz funcional), por lo que ya sabemos todo lo necesario. Solo necesito la implementación del método operacion.

Tenemos que ver la expresión como un objeto de una clase anónima que implementara la interfaz. Es por ello que podremos guardar el resultado de la lambda en una referencia, ya que en el fondo no deja de ser un objeto como el que obteníamos con la clase completa anterior. Simplemente hemos simplificado la sintaxis.

Según las necesidades tendremos diferentes formas de escribir una lambda. Por ejemplo:

- **Sin parámetros:** Los paréntesis son obligatorios y van vacíos.
`() -> System.out.println("Hola");`
- **Un solo parámetro:** Se pueden omitir los paréntesis por comodidad.
`p -> p.getNombre();`
- **Varias líneas de código:** Las llaves `{ }` y la palabra reservada `return` (si devuelve un valor) vuelven a ser obligatorias.
`(a, b) -> { int suma = a + b; return suma; }`

Concepto clave (Effectively final):

- **Lo que ocurre:** Una expresión lambda puede acceder a variables locales que están definidas fuera de ella (en su ámbito envolvente), pero solo puede leerlas.
- **La restricción:** Por definición de la programación funcional en Java, estas variables externas deben ser **finales de forma efectiva** (*effectively final*). Esto significa que **no pueden ser modificadas** dentro del cuerpo de la lambda ni reasignadas después de su inicialización fuera de ella.
- **Por qué:** Esta regla protege el principio de **no-efectos secundarios** (no *side-effects*) y asegura que el estado no cambie, un pilar de la programación funcional.

Por tanto, Si intentas reasignar o cambiar el estado de una variable externa dentro de la lambda, el compilador lo impedirá. Por ejemplo, el siguiente código compilará y funcionará:

```
public class Programa {
    public static void main(String[] args) {
        int c = 2;
        Matematica sumar = (a, b) -> (a + b) * c;
        System.out.println(sumar.operacion(3, 4));
    }
}
```

Sin embargo, esto dará un error de compilación (Local variable c defined in an enclosing scope must be final or effectively final), ya que estamos intentando modificar una variable externa a la lambda:

```
public class Programa {
    public static void main(String[] args) {
        int c;
        Matematica sumar = (a, b) -> {
            c = 2;
            return (a + b) * c;
        };
        System.out.println(sumar.operacion(3, 4));
    }
}
```

Al ser objetos, podemos incluso crear colecciones de objetos que implemente el método de forma diferente, recorriendo posteriormente la colección y ejecutando el método para ver diferentes resultados provenientes de diferentes implementaciones:

```
public class Programa {
    public static void main(String[] args) {
        Matematica sumar = (a, b) -> a + b;
        Matematica restar = (a, b) -> a - b;
        Matematica multiplicar = (a, b) -> a * b;
        Matematica dividir = (a, b) -> a / b; //OJO, no
        hemos controlado division by zero

        List<Matematica> operaciones = new ArrayList<>();
        operaciones.add(sumar);
        operaciones.add(restar);
        operaciones.add(multiplicar);
        operaciones.add(dividir);

        for (Matematica operacion : operaciones) {
            System.out.println(operacion.operacion(6, 2));
        }
    }
}
```

Pero, si la colección operaciones ya espera objetos de tipo Matematica, ¿Para qué declarar los objetos? Podemos añadir estos objetos directamente a la colección a través de la lambda:

```
public class Ejemplo06 {
    public static void main(String[] args) {
        List<Matematica> operaciones = new ArrayList<>();
        operaciones.add((a, b) -> a + b);
        operaciones.add((a, b) -> a - b);
        operaciones.add((a, b) -> a * b);
        operaciones.add((a, b) -> a / b);

        for (Matematica operacion : operaciones) {
            System.out.println(operacion.operacion(6, 2));
        }
    }
}
```

Generará la salida:

```
8
4
12
3
```

¿Cómo queda nuestro objeto Comparator con estos cambios? Recordemos que partimos de esto:

```
new Comparator<Persona>() {
    @Override
    public int compare(Persona p1, Persona p2) {
        return Integer.compare(p1.getEdad(),
p2.getEdad());
    }
}
```

Comparator es una interfaz funcional con un único método abstracto: compare.

Por otro lado, el método sort de List ya espera un objeto Comparator, y la colección ya está asociada a un tipo (Persona en este caso), por lo que nos podemos ahorrar todo eso.

También conocemos los tipos que espera compare, y el cuerpo solo tiene una línea, por lo que nos ahorramos el return. Nos queda entonces:

```
public class Principal {
    public static void main(String[] args) {
        Persona personaA = new Persona("pedro", "perez", 20);
        Persona personaB = new Persona("ana", "blanco", 15);
        Persona personaC = new Persona("miguel", "alvarez",
50);

        List<Persona> lista = new ArrayList<>();
        lista.add(personaA);
        lista.add(personaB);
        lista.add(personaC);

        lista.sort((p1, p2) -> Integer.compare(p1.getEdad(),
p2.getEdad()));

        for (Persona p: lista) {
            System.out.println(p.getNombre() + " , " +
p.getEdad());
        }
    }
}
```

Incluso, la interfaz `Comparator` incluye desde Java 8 métodos de ayuda muy potentes como `comparing`, lo que nos permite dejar nuestra expresión lambda así (son métodos estáticos o default, que no chocan con la filosofía de las interfaces funcionales):

```
lista.sort(Comparator.comparing(p -> p.getEdad()));
```

Vemos por tanto que la creación del objeto tipo `Comparator` queda reducida a su mínima expresión. Hemos reducido drásticamente el código superfluo, pero la evolución de Java no se detiene aquí. ¿Podemos hacerlo aún más simple? La respuesta es sí, gracias a las referencias a métodos. El uso de Java lambdas es cada día más importante para conseguir un código más compacto y más sencillo. Recordemos, además, que muchos de los lenguajes modernos hacen uso intensivo de estos conceptos.

4.METHOD REFERENCES

Las referencias a métodos (method references) son un tipo especial de expresiones lambda. Se suelen usar para crear expresiones lambda sencillas haciendo referencia a métodos existentes.

Existen cuatro tipos de referencias de métodos:

- Métodos estáticos
- Métodos de instancia de objetos particulares
- Métodos de instancia de un objeto arbitrario de un tipo particular.
- Constructor

En este capítulo exploraremos las referencias a métodos en Java.

IMPORTANTE: Las Method References (`::`) son "todo o nada". Solo sirven como atajo si lo **único** que vas a hacer dentro del paréntesis es **llamar a un método preexistente**.

En el momento en que necesitas añadir la más mínima lógica extra (concatenar un texto, sumar un +1, hacer un pequeño condicional), **tienes que volver a la sintaxis Lambda**.

4.1 REFERENCIA A UN MÉTODO ESTÁTICO (Clase::metodoEstatico)

Se usa cuando la lambda simplemente llama a un método estático de una clase.

- **Con Lambda:** `(a, b) -> Math.max(a, b)`
- **Con Method Reference:** `Math::max`

```

public class MethodReferences {
    public static void main(String[] args) {
        // Ejemplo: Guardar en una variable la lógica para
        // convertir un String a Integer
        // Usamos la interfaz funcional Function<T, R>
        // (Recibe T, devuelve R)
        // 1. Usando lambda
        Function<String, Integer> conversorLambda = s ->
        Integer.parseInt(s);
        // 2. Usando Method Reference
        Function<String, Integer> conversorRef =
        Integer::parseInt;
        // Aplicamos la lógica
        int numero = conversorRef.apply("123");
    }
}

```

4.2 REFERENCIA A UN MÉTODO DE INSTANCIA DE UN OBJETO PARTICULAR (objeto::metodoInstancia)

Se usa cuando llamamos a un método de un objeto que ya ha sido creado y guardado en una variable (o un objeto estático ya existente). El caso más famoso es el de imprimir por pantalla.

- **Con Lambda:** texto -> System.out.println(texto)
- **Con Method Reference:** System.out::println

```

public class MethodReferences {
    public static void main(String[] args) {

        List<Persona> lista = new ArrayList<>();
        lista.add(new Persona("Ana", "Blanco", 15));
        // Usando Method Reference sobre el objeto
        // preexistente 'System.out'
        lista.forEach(System.out::println);
    }
}

```

4.3 REFERENCIA A UN MÉTODO DE INSTANCIA DE UN OBJETO ARBITRARIO DE UN TIPO PARTICULAR (Clase::metodoInstancia)

Este es el tipo más utilizado al trabajar con nuestras propias clases (como Persona). Se usa cuando el propio parámetro que entra en la lambda es el que va a invocar a su propio método.

- **Con Lambda:** `p -> p.getNombre()`
- **Con Method Reference:** `Persona::getNombre`

Veamos cómo queda la ordenación por edad de nuestra lista de personas utilizando esta técnica (este es el estándar actual en la industria):

```
public class MethodReferences2 {
    public static void main(String[] args) {
        Persona personaA = new Persona("pedro", "perez",
20);
        Persona personaB = new Persona("ana", "blanco", 15);
        Persona personaC = new Persona("miguel", "alvarez",
50);

        List<Persona> lista = new ArrayList<>();
        lista.add(personaA);
        lista.add(personaB);
        lista.add(personaC);

        // REFERENCIA A UN MÉTODO DE INSTANCIA DE UN OBJETO
        // ARBITRARIO DE UN TIPO
        // PARTICULAR
        // 1. Con Lambda:
        lista.sort(Comparator.comparingInt(p ->
p.getEdad()));
        // 2. Con un method reference
        lista.sort(Comparator.comparing(Persona::getEdad));
        lista.forEach(System.out::println);
    }
}
```

4.4 REFERENCIA A UN CONSTRUCTOR (Clase::new)

Se usa cuando la lambda solo se encarga de crear un nuevo objeto usando new.

- **Con Lambda:** `() -> new ArrayList<>()`
- **Con Method Reference:** `ArrayList::new`

```
public class MethodReferences2 {
    public static void main(String[] args) {
        // Ejemplo con la interfaz funcional Supplier
        (proveedor)
        // Nota: requiere que Persona tenga un constructor
        sin parámetros
        Supplier<Persona> creadorLambda = () -> new
        Persona();
        Supplier<Persona> creadorReferencia = Persona::new;
        // Usamos la referencia para fabricar un objeto
        nuevo
        Persona nuevaPersona = creadorReferencia.get();
        Persona personaA = new Persona("pedro", "perez",
        20);
        Persona personaB = new Persona("ana", "blanco", 15);
        Persona personaC = new Persona("miguel", "alvarez",
        50);

        List<Persona> lista = new ArrayList<>();
        lista.add(personaA);
        lista.add(personaB);
        lista.add(personaC);

        // REFERENCIA A UN MÉTODO DE INSTANCIA DE UN OBJETO
        ARBITRARIO DE UN TIPO
        // PARTICULAR
        // 1. Con Lambda:
        lista.sort(Comparator.comparingInt(p ->
        p.getEdad()));
        // 2. Con un method reference
        lista.sort(Comparator.comparing(Persona::getEdad));
        lista.forEach(System.out::println);
    }
}
```

5. PRINCIPALES INTERFACES FUNCIONALES DE LA API Java

Java 8 introdujo el paquete `java.util.function`, que contiene docenas de interfaces funcionales listas para usar. Sin embargo, para dominar el procesamiento de datos y las colecciones (lo que veremos a continuación con los Streams), basta con comprender profundamente tres de ellas.

Cada una representa un concepto lógico fundamental: **decidir**, **transformar** y **actuar**.

5.1 Predicate<T>

Se utiliza para tomar decisiones. Representa una función que recibe un objeto y devuelve un valor booleano (true o false). Es el equivalente a una pregunta de "sí o no".

- **Método abstracto:** boolean test(T t)
- **Uso principal:** Filtrar colecciones o validar datos (ej. método filter).

5.2 Function<T, R>

Se utiliza para hacer transformaciones. Representa una función que recibe un objeto de tipo T, hace algo con él, y devuelve un objeto de tipo R (que puede ser del mismo tipo o de uno completamente distinto).

- **Método abstracto:** R apply(T t)
- **Uso principal:** Extraer información de un objeto o convertir un tipo de dato en otro (ej. método map).

Java

```
// Ejemplo: Dada una Persona (T), devuelve su nombre en formato String (R)
Function<Persona, String> extraerNombre = Persona::getNombre;
```

```
String nombre = extraerNombre.apply(new Persona("Ana", 25)); // Devuelve "Ana"
```

5.3 Consumer<T>

Se utiliza para ejecutar una acción. Representa una operación que recibe un objeto, realiza una acción con él, pero no devuelve nada (void). Es un "callejón sin salida" para los datos.

- **Método abstracto:** void accept(T t)
- **Uso principal:** Imprimir por pantalla, guardar en base de datos o modificar estados externos (ej. método forEach).

Veamos un ejemplo de uso de las tres interfaces funcionales:

```
public class Ejemplo09 {
    public static void main(String[] args) {
        Persona persona = new Persona("Pedro", "Pérez", 20);
        // Ejemplo Predicate: ¿Es la persona mayor de edad?
        Predicate<Persona> esMayorEdad = p -> p.getEdad() >=
18;

        if(esMayorEdad.test(persona)) { // El método test
devuelve true
    }
}
```

```

        System.out.println("Es mayor de edad");
    } else {
        System.out.println("Es menor de edad");
    }

    // Ejemplo Function: Dada una Persona (T), devuelve
    su nombre en formato String (R)
    //Function<Persona, String> extraerNombre = (p) ->
    p.getNombre();
    Function<Persona, String> extraerNombre =
    Persona::getNombre;
    System.out.println(extraerNombre.apply(persona)); //
    Devuelve "Pedro"

    // Ejemplo Consumer: Recibe una Persona y la imprime
    por consola
    Consumer<Persona> imprimirPersona =
    System.out::println;
    imprimirPersona.accept(persona); // Imprime el
    objeto por pantalla
    }
}

```

Resumen para recordar:

- ¿Quieres hacer una pregunta? -> Usa un Predicate.
- ¿Quieres cambiar de forma? -> Usa una Function.
- ¿Quieres consumir el dato sin devolver nada? -> Usa un Consumer.

Veamos ahora otro ejemplo en el que, utilizando la API estándar de Java, se usan algunas de estas interfaces para resolver problemas de forma eficiente:

Antes de Java 8, para borrar elementos de una lista bajo una condición sin que el programa fallara con una `ConcurrentModificationException`, había que instanciar un `Iterator` y montar un bucle. Ahora, la lista se limpia a sí misma inyectándole un `Predicate` a través del método `removeIf` de `Collection`.

```

public static void main(String[] args) {
    Persona p1 = new Persona("pedro", "perez", 20);
    Persona p2 = new Persona("ana", "blanco", 15);
    Persona p3 = new Persona("miguel", "alvarez", 50);

    List<Persona> lista = new ArrayList<>();
    lista.add(p1);
    lista.add(p2);
    lista.add(p3);
}

```

```
// Eliminamos los menores de edad mediante un filtro
// Si la lambda devuelve 'true', el elemento se "destruye".
lista.removeIf(p -> p.getEdad() < 18);
lista.forEach(System.out::println);
}
```

Intenta cambiar la expresión lambda de `removeIf` por una referencia a un método. Para ello, crea en la clase `Persona` un método `esMenorEdad()` que devuelva `true` si la persona tiene menos de 18 años y `false` en caso contrario.

6.LA API Stream

Hasta ahora, si queríamos procesar una lista de personas, usábamos bucles `for` y condicionales `if`. Esto es programación imperativa: le decimos a Java exactamente cómo hacer cada paso.

La API Stream (introducida en Java 8) nos permite usar programación declarativa: le decimos a Java qué queremos conseguir, y él se encarga del cómo.

Un Stream es como una cinta transportadora. Los datos entran por un lado, pasan por diferentes máquinas que los transforman o filtran, y salen por el otro lado procesados.

Para nuestros ejemplos utilizaremos colecciones, ya que es uno de los objetos donde más claramente se pueden aplicar. Todo pipeline (tubería) de un Stream consta de 3 fases obligatorias:

6.1 CREACIÓN DEL OBJETO Stream

No podemos usar métodos de Stream directamente sobre una colección. Cuando Java 8 introdujo los Streams, no fue modificando cada lista individualmente, sino que añadieron `.stream()` como un *default method* directamente en la interfaz raíz `Collection`.

Esto significa que estas tres grandes familias lo tienen disponible por defecto:

- **List:** `ArrayList`, `LinkedList`, `Vector`, etc.
- **Set:** `HashSet`, `TreeSet`, `LinkedHashSet`, etc.
- **Queue y Deque:** `PriorityQueue`, `ArrayDeque`, etc.

¿Qué ocurre con los mapas? Clases como `HashMap` o `TreeMap` **NO** tienen el método `.stream()`. El motivo arquitectónico es que la interfaz `Map` no hereda de `Collection` (porque un mapa no es una colección de elementos simples, sino de pares clave-valor).

¿Cómo se usa un Stream con un Map entonces? Hay que extraer primero una "vista" del mapa que sí sea una Collection. Se puede hacer de tres formas, dependiendo de qué datos necesites procesar:

- **Sobre los valores:** `map.values().stream()`
- **Sobre las claves:** `map.keySet().stream()`
- **Sobre los pares completos (Clave + Valor):** `map.entrySet().stream()`

Este último (`entrySet`) es el más común cuando necesitas tanto la clave como el valor en tu flujo funcional, ya que te devuelve un Stream de objetos `Map.Entry<K, V>`.

Primero debemos convertir la colección en un flujo de datos usando el método `.stream()`.

```
List<Persona> lista = ... // Creamos la lista
lista.stream(); // A partir de aquí, tenemos un Stream de
objetos Persona
```

6.2 OPERACIONES INTERMEDIAS

Son métodos que procesan los datos y **devuelven otro Stream**, por lo que podemos encadenar varios. Aquí es donde brillan las interfaces funcionales que hemos visto. Son "perezosas" (*lazy*): no hacen nada hasta que se llega a la fase final.

- **filter(Predicate<T>)**: Deja pasar solo los elementos que cumplan una condición.
- **map(Function<T, R>)**: Transforma cada elemento en otra cosa.
- **sorted()** / **sorted(Comparator<T>)**: Ordena los elementos en el flujo.

6.3 OPERACIÓN TERMINAL

Es el final de la cinta transportadora. Activan todo el proceso y devuelven un resultado final (no un Stream). Una vez ejecutada, **el Stream se consume y no puede volver a usarse**.

- **forEach(Consumer<T>)**: Ejecuta una acción por cada elemento final (ej. imprimir).
- **collect(Collector)**: Empaqueta el resultado de nuevo en una lista, conjunto, etc.
- **count()**: Cuenta cuántos elementos han sobrevivido al flujo.

6.4 EJEMPLOS

Ejemplo 1: Partimos de una colección de objetos de la clase `Persona`. Crearemos una nueva colección a partir de ella, en este caso de Strings compuestas por la concatenación

del nombre y apellidos de la persona en mayúsculas, filtrando para quedarnos con los mayores de edad.

```
public class Ejemplo10 {
    public static void main(String[] args) {

        List<Persona> lista = List.of(
            new Persona("pedro", "perez", 20),
            new Persona("ana", "blanco", 15),
            new Persona("miguel", "alvarez", 50));

        List<String> nombresMayores = lista.stream() // A partir de
aquí, tenemos un Stream de objetos Persona
            .filter(p -> p.getEdad() >= 18) // Nos quedamos con los
mayores de edad

            .sorted(Comparator.comparing(Persona::getApellidos).thenComparing(Persona::getNombre)) // Ordenamos por apellidos y nombre
            .map(p -> p.getApellidos() + ", " + p.getNombre())
            .map(String::toUpperCase)
            .collect(Collectors.toList());

        nombresMayores.forEach(System.out::println);
    }
}
```

La clase Stream tiene un método final `toList()` que acumula los elementos del stream en una lista de tipo List, que es inmutable. En la mayoría de los casos, si necesitamos una lista, es preferible este método porque es más claro, más seguro y más “idiomático”. Usaremos `collect(...)` cuando necesitemos algo distinto: **otro tipo de colección, una colección mutable**, o bien operaciones más avanzadas como **agrupaciones, particiones o reducciones**.

Regla de oro de la inmutabilidad: Los Streams **nunca** modifican la lista original. Nuestro objeto inicial, por ejemplo una lista de personas, sigue intacto. El Stream ha generado una colección completamente nueva.

Ejemplo 2: Veamos ahora un ejemplo en el que tenemos un archivo llamado `ventas.txt` que contiene miles de líneas. Queremos saber **cuántas ventas de la categoría "Tecnología" superan los 500€**.

Para conectar nuestro conocimiento clásico de lectura de archivos con el nuevo mundo funcional, utilizaremos el método `.lines()` de la clase `BufferedReader`.

```

public class Ejemplo11 {
    public static void main(String[] args) {

        // Usamos el clásico try-with-resources con las clases
        de java.io que ya conocemos
        try (BufferedReader br = new BufferedReader(new
        FileReader("ventas.txt"))) {

            // El puente: br.lines() convierte el lector
            tradicional en un Stream<String>
            long ventasAltas = br.lines()
                .map(String::trim) //
                Limpiamos espacios en blanco
                .filter(l -> !l.isEmpty()) // Saltamos
                líneas vacías
                .filter(l -> l.contains("TEC")) // Buscamos
                la categoría Tecnología
                .map(l -> l.split(";")[1]) //
                Suponiendo formato "CATEGORIA;PRECIO"
                .mapToDouble(Double::parseDouble) //
                Convertimos el precio a número
                .filter(precio -> precio > 500) //
                Filtramos por importe
                .count(); // Contamos
                el resultado final
            System.out.println("Ventas tecnológicas > 500€: " +
            ventasAltas);
        } catch (IOException e) {
            System.err.println("Error al leer el archivo: " +
            e.getMessage());
        }
    }
}

```

Otras aplicaciones rápidas de los Streams::

- **Bases de datos:** Muchas librerías modernas (como Hibernate o Spring Data) permiten obtener resultados de una consulta directamente como un Stream.
- **Peticiones HTTP:** Procesar la respuesta de una API JSON línea a línea.
- **Directorios:** Files.list(path) devuelve un Stream con todos los archivos de una carpeta para filtrarlos por extensión o tamaño.